

The Rational Unified Process

The Unified Process

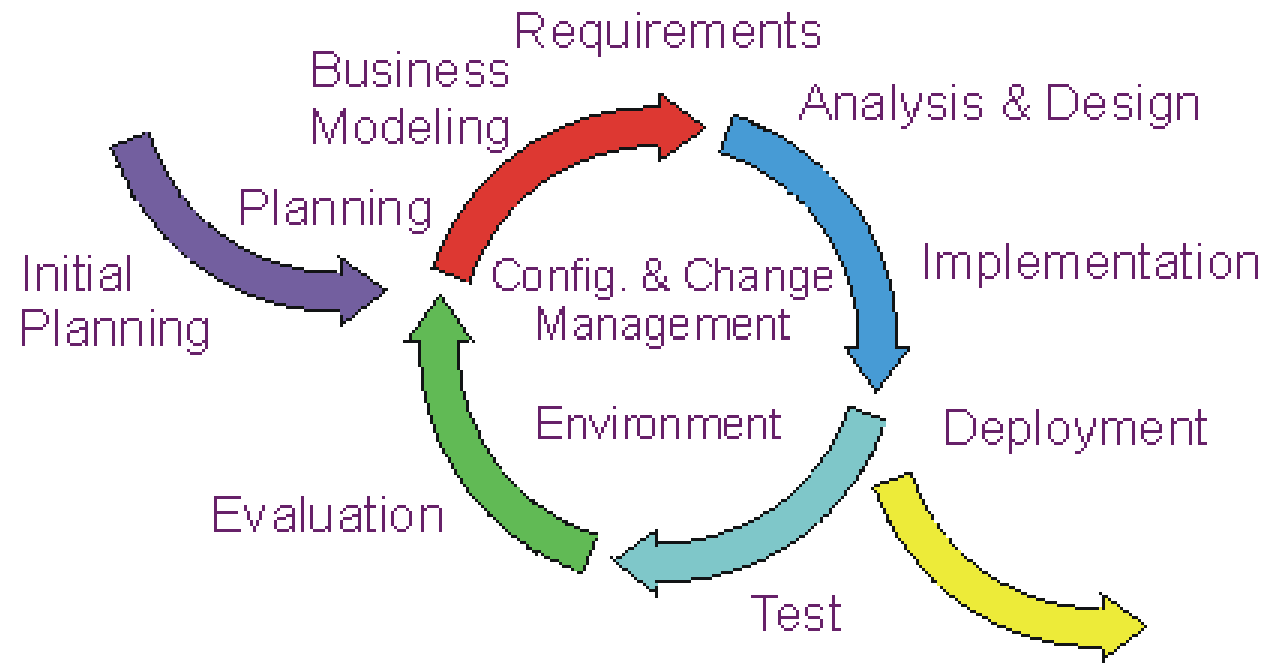
- Three of the most successful object-oriented methodologies were
 - Booch's method
 - Jacobson's Objectory
 - Rumbaugh's OMT

Software Lifecycles

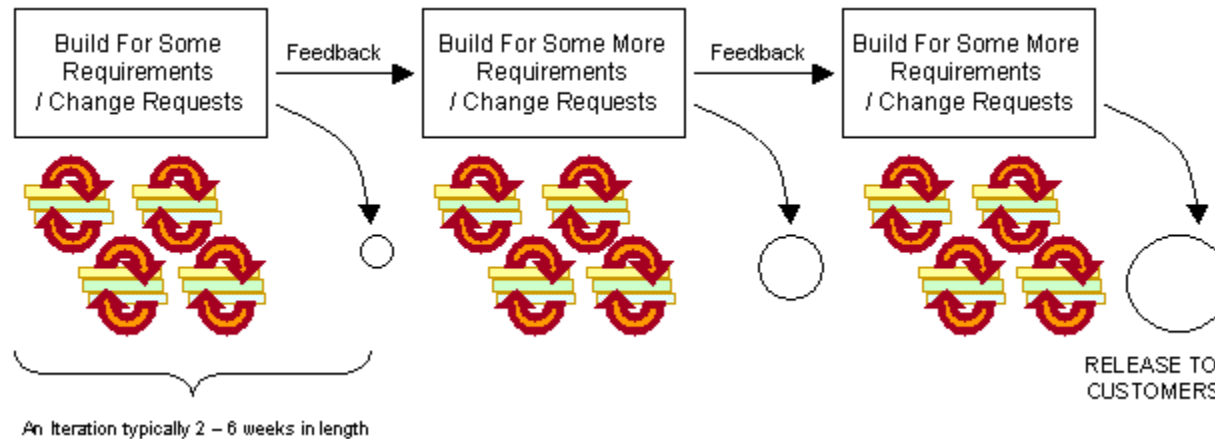
- Fundamental processes - the Development Lifecycle
 - Waterfall
 - Iterative
 - Spiral
 - Rational Unified Process (RUP)
 - Extreme Programming



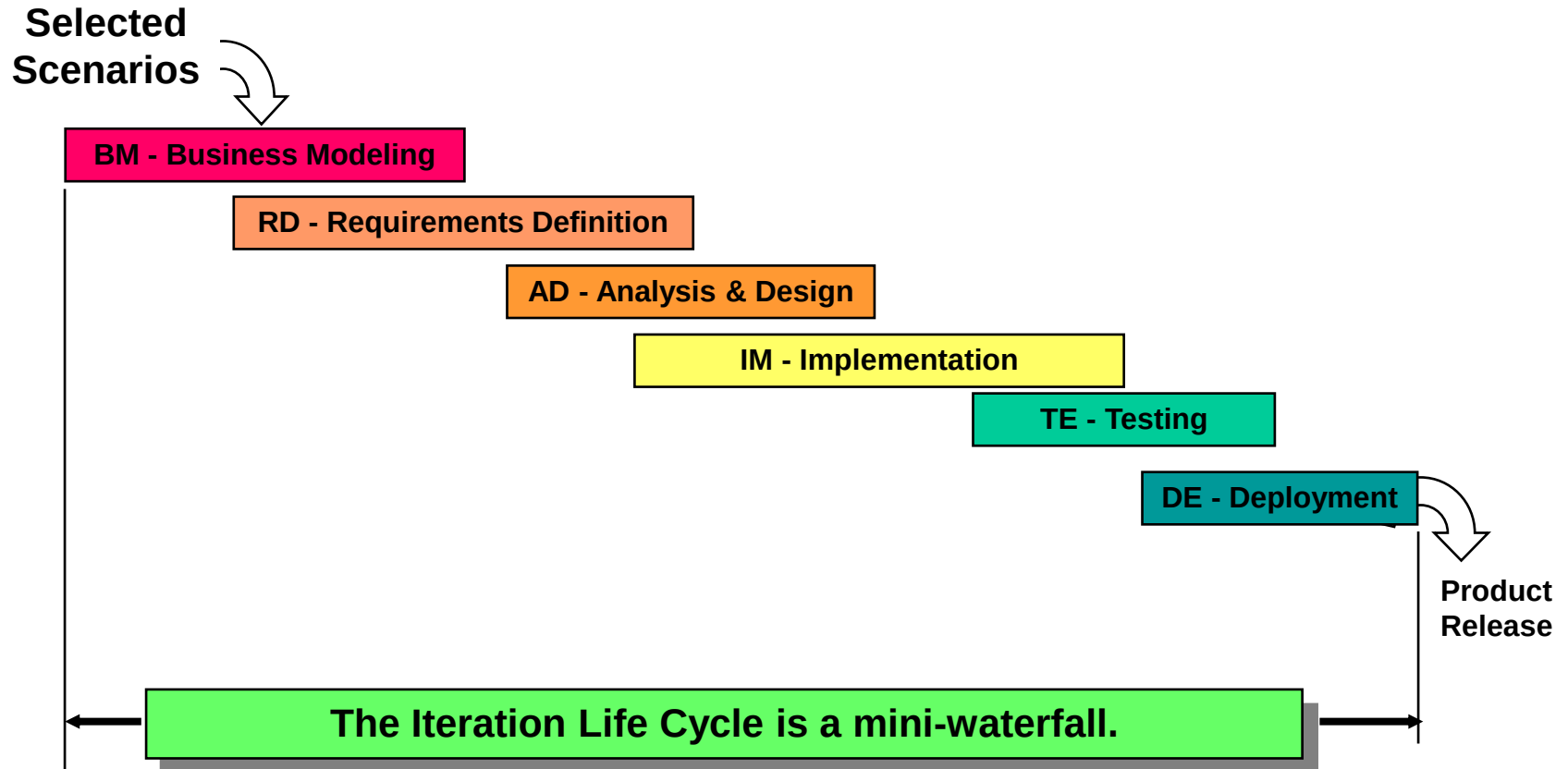
Iterative Development



Iterative Development

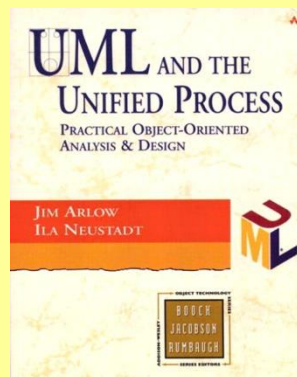


Each Iteration follows a mini-Waterfall



The Unified Process (contd)

- In 1999, Booch, Jacobson, and Rumbaugh published a complete object-oriented analysis and design methodology that unified their three separate methodologies
 - Original name: *Rational Unified Process* (RUP)
 - Next name: *Unified Software Development Process* (USDP)
 - Name used today: *Unified Process* (for brevity)
 - BUP
 - OpenUP



The Unified Process (contd)

- The Unified Process is *not* a series of steps for constructing a software product
 - No such single "one size fits all" methodology could exist
 - There is a wide variety of different types of software
- The Unified Process is an adaptable methodology
 - It has to be modified for the specific software product to be developed

Rational Unified Process (RUP)

What is the Rational Unified Process?

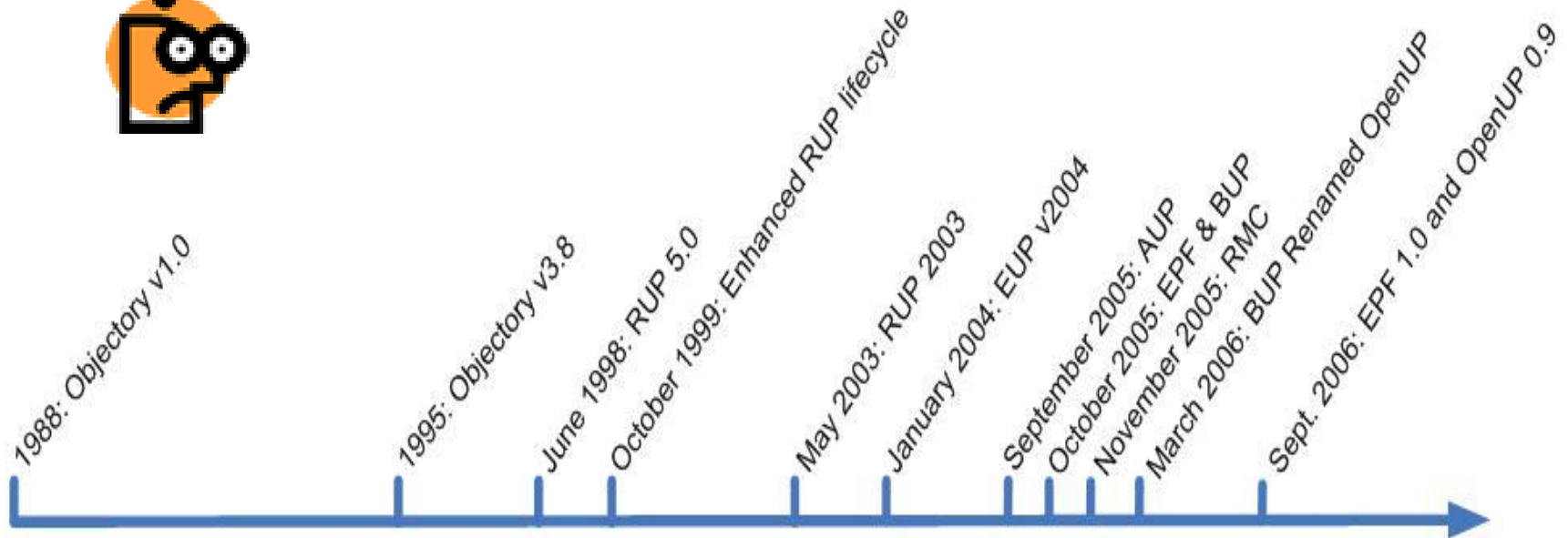
- “Process Product” developed by Rational Software and integrated with its suite of software development tools
- The Rational Unified Process is a *software engineering process*, provides a disciplined approach to assigning tasks and responsibilities within a development organization.



RUP- History

- The Rational Approach was developed at Rational Software in the 1980s and 1990s
- In 1995 Rational Software acquired the Swedish Company Objectory AB.
- The Rational Unified Process was the result of the merger of the Rational Approach and the Objectory process developed by Objectory founder Ivar Jacobson.
- The first version of the Rational Unified Process, version 5.0, was released in 1998.

RUP- Genealogy



Rational Unified Process (RUP)

What is its Goal?

To ensure the production of high-quality software that meets the needs of its end users within

- predictable schedule
- budget



RUP as a “Process Framework”

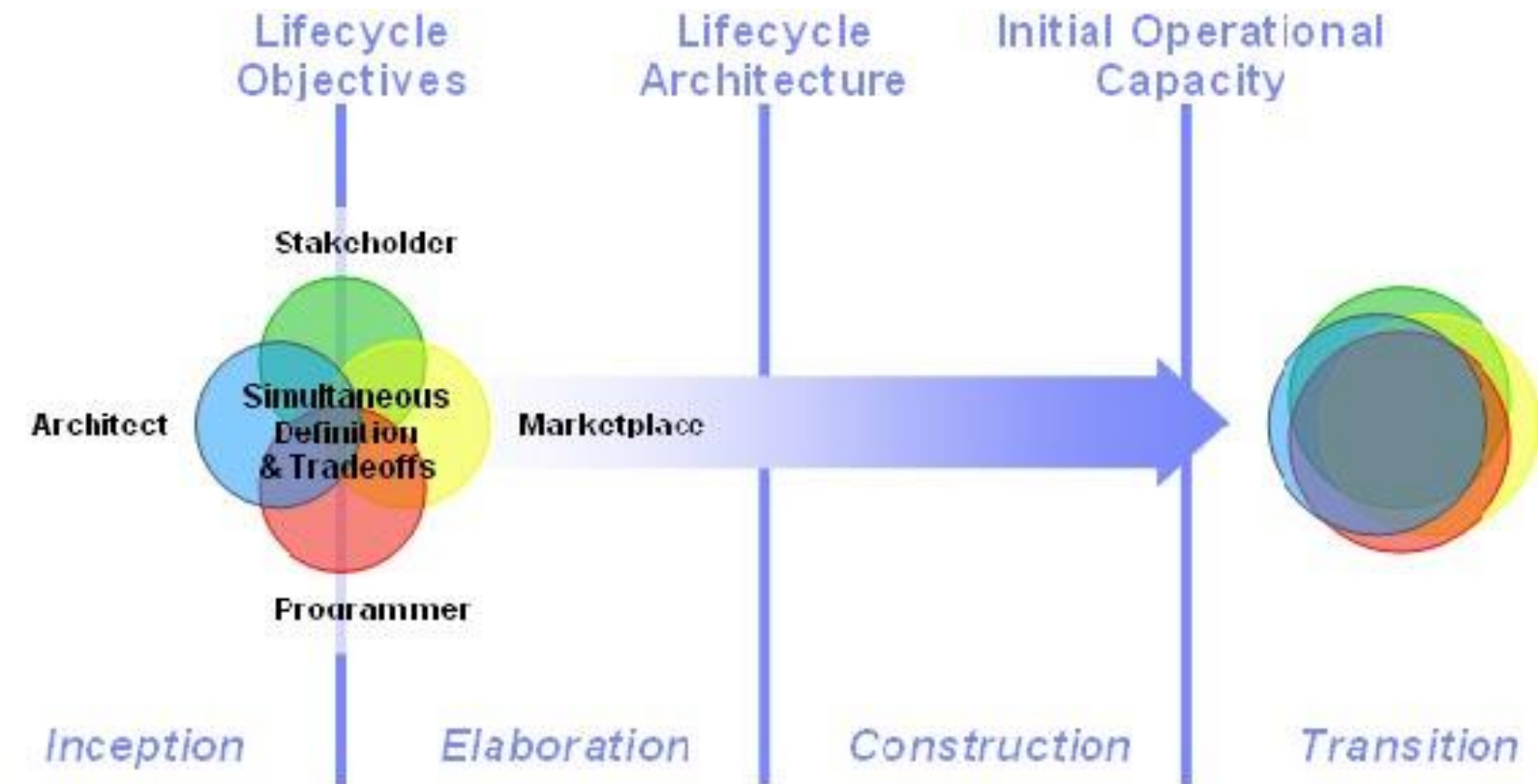
- RUP as a “Process Framework” is extensible and has the best practices:
 - Iterative Development
 - Requirements Management
 - Component-Based Architectural Vision
 - Visual Modeling of Systems
 - Quality Management
 - Change Control Management

RUP as a “Product”

- RUP as a “Product” has features:
 - Online Repository of Process Information and Description in HTML format
 - Templates for all major artifacts, including:
 - Requisite-Pro templates (requirements tracking)
 - Word Templates for Use Cases
 - Project Templates for Project Management
 - Process Manuals describing key processes



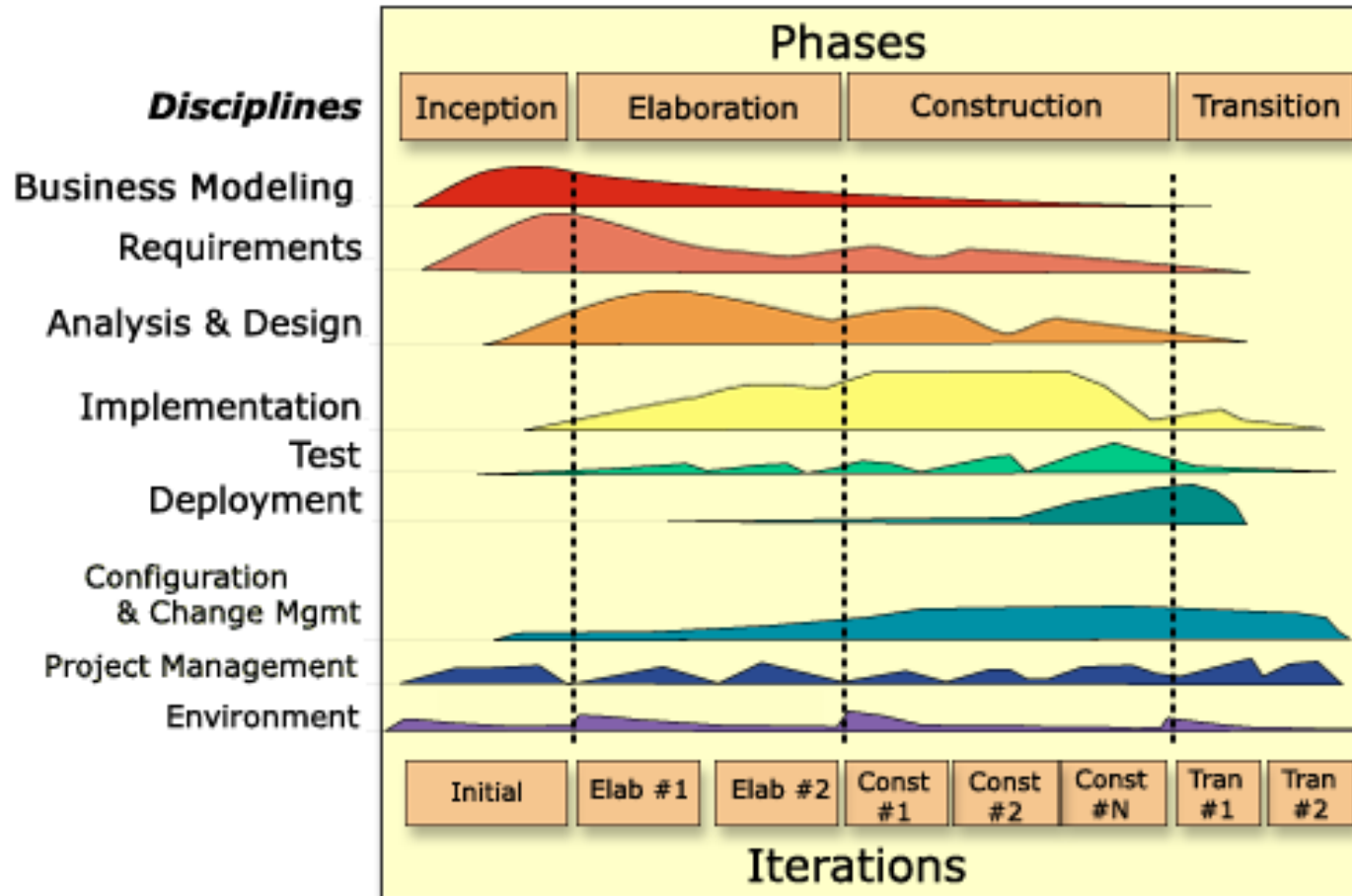
RUP Roadmap



Source: SEI EPIC

RUP Architecture

Process Structure – two Dimensions



RUP Architecture

- The process has two Dimensions:

- [Dynamic] The horizontal axis represents time and shows the lifecycle of the process. It represents the dynamic aspect of the process and it is expressed in terms of cycles, phases, iterations, and milestones.

- [Static] The vertical axis represents core process disciplines. It represents the static aspect of the process: its description in terms of process components, activities, disciplines, artifacts, and roles

Why a Two-Dimensional Model?

- The Unified Process handles the inevitable changes well
 - The moving target problem
 - The inevitable mistakes
- The Unified Process is the best solution found to date for treating a large problem as a set of smaller, largely independent subproblems
 - It provides a framework for incrementation and iteration
 - In the future, it will inevitably be superseded by some better methodology

RUP Feature – Iterative Development

An Iterative Development Process

- Recognizes the reality of changing requirements
40% of final requirements arrived after the analysis phase, after development had already begun
- Promotes early risk mitigation, by breaking down the system into mini-projects and focusing on the riskier elements first
- Allows to “plan a little, design a little, and code a little”
- All participants, are involved early in the project
- Allows the process itself to modulate with each iteration, allowing you to correct errors sooner and put into practice lessons learned in the prior iteration

RUP Feature – Iterative Development

An Incremental Development Process...

- Allows for software to evolve, not be produced in one huge effort
- Allows software to improve, by giving enough time to the evolutionary process itself
- Forces attention on stability; supports multiple additions
- Allows the system (a small subset of it) to actually run much sooner than with other processes
- Allows for the management of risk, by exposing problems earlier on in the development process

RUP Feature – Iterative Development

Goals of Iteration Process:

- The primary goal of each iteration is to slowly look at the risk facing the project, namely:
 - performance risks
 - integration risks
 - conceptual risks
- Each iteration is risk-driven
- The result of a single iteration is an increment--an incremental improvement of the system, yielding an evolutionary approach

RUP Feature – Requirements Management

Why too much emphasis on requirements?

■ Example of a vague requirement:

➤ “A part record and a plant record are read from the database. If **it** contains the letter A directly followed by the letter Q, then calculate the cost of transporting that part to that plant”

➤ To what does **it** refer?

- The part record?
- The plant record?
- Or the database?



RUP Feature – Requirements Management

- Requirements management: a systematic approach to capture, communicate, and manage requirements.
- What are the benefits of requirements management ?

RUP Feature – Requirements Management

- **Better control of complex projects**

Vague requirements and lack of understanding of the intended system behavior are typical examples in out-of-control projects.

- **Improved software quality and customer satisfaction**

All product stakeholders have a common understanding on how the software product should be built and tested.

- **Reduced project costs and delays**

Validate and verify requirements in early stages of the project cuts project cost and avoids schedule overruns.

- **Improved team communication**

Well-managed requirements build a common understanding of the project needs and commitments among the stakeholders: users, customers, management, designers, and testers.

Component Based Development

Component-based development :



- *What does it mean modular architecture/Design?*
 - *Well-formed components that can be designed and tested individually and integrated gradually to form the whole system.*
- *What does it mean invasive changes vs. none-invasive changes?*
- *Components can be developed to be reusable across the organization, which yields to improved productivity and quality.*
- *Shifting developers' focus from writing code to product assembly from components.*

RUP Feature – Component Based Development

RUP supports for component-based development:

- *The use of the iterative approach to identify components progressively and decide which ones to develop, which ones to reuse, and which ones to buy.*
- *The focus on software architecture which articulates the structure and the way they interact.*
- *It utilizes the concepts of packages, subsystems, and layers in the analysis and design phase to organize components and specify interfaces.*
- *Component based testing. Testing is organized around single components first and then is gradually expanded to include larger sets of integrated components*

RUP Feature – Component Based Development

Better testing:

- Each component is tested as soon as it has been implemented
 - *Unit testing*
- At the end of each iteration, the completed components are combined and tested
 - *Integration testing*
- When the product appears to be complete, it is tested as a whole
 - *Product testing*
- Once the completed product has been installed on the client's computer, the client tests it
 - *Acceptance testing*

Unified Modeling Language (UML)

■ What is UML?

- The Unified Modeling Language (UML) is a **modeling** language for visualizing, specifying, constructing, and documenting the artifacts of a software system.
- UML gives a standard means of writing the system's blueprints, covering conceptual items such as business processes and system functions as well as concrete items such as classes written in a specific programming language, database schemas, and reusable software components



RUP Feature –Visually model Software

- The UML is a modeling language to express the various models, but it does not tell you how to develop software.
- UML provides the annotations and mechanisms, but it doesn't tell you how to write the artifacts.
- *Rational Unified Process hand-in-hand with the UML will guide the engineer to the effective use of the UML for modeling.*

RUP Feature –Configuration and Change Management

- In all software projects, change is inevitable.
- RUP defines methods to control, track and monitor changes.
 - Changes made to:
 - Requirements
 - Design
 - Implementation
- Track defects
- Keep artifacts in synch with software releases

RUP –Other Key Features

- The role of use cases in driving many aspects of the development
(Use-Case-Driven Development)
- Its use as a process framework that can be tailored and extended by an adopting organization
(Process Configuration)
- The need for software development tools to support the process
(Tools Support)

RUP-Other Key Features

Use-Case-Driven Development

- Use cases defined are the foundation for the of development process.
- Use cases are used in the process disciplines:
 - business modeling,
 - requirements,
 - design,
 - test,
 - management.

RUP-Other Key Features

Process Configuration

- The Rational Unified Process is a process framework
- The adopting organization can modify, adjust, and expand RUP to accommodate the organization's needs
- *RUP 2003* offers two process bases—one for small projects and one for larger and more formal projects

RUP-Other Key Features

Tools Support

- A good tool set is necessary for effective processes.
- RUP is supported by a list of tools that automate steps in many activities.
- These tools are used to create and maintain the various artifacts
- The tools are crucial in supporting change and configuration management for each iteration.

RUP - Overview

Rational Unified Process®

File Edit View Favorites Tools Help Address C:\temp\Classic\index.htm

Search Print

Who Am I Tree Sub

Team

Getting Started Role Sets Delivery Processes

Welcome

Welcome to the Rational Unified Process (RUP). RUP provides best practices and guidance for successful software development.

Expand All Sections Collapse All Sections

Main Description

Learning

- Getting Started
- How to Navigate
- Key Principles For Business-Driven Development

Navigation Links

- Roles
- Work Products (by domain)
- Processes

Disciplines

- Business Modeling
- Requirements
- Analysis & Design
- Implementation
- Test
- Deployment
- Configuration & Change Mgmt
- Project Management
- Environment

Resources

- Overview
- RUP on developerWorks
- Training
- Rational Method Composer
- The Rational Edge

Phases

	Inception	Elaboration	Construction	Transition
Business Modeling	[Red area]			
Requirements	[Red area]			
Analysis & Design	[Yellow area]			
Implementation	[Yellow area]			
Test	[Green area]			
Deployment	[Green area]			
Configuration & Change Mgmt	[Green area]			
Project Management	[Blue area]			
Environment	[Blue area]			

Iterations

	Initial	E1	E2	C1	C2	CM	T1	T2
Business Modeling	[Red area]							
Requirements	[Red area]							
Analysis & Design	[Yellow area]							
Implementation	[Yellow area]							
Test	[Green area]							
Deployment	[Green area]							
Configuration & Change Mgmt	[Green area]							
Project Management	[Blue area]							
Environment	[Blue area]							

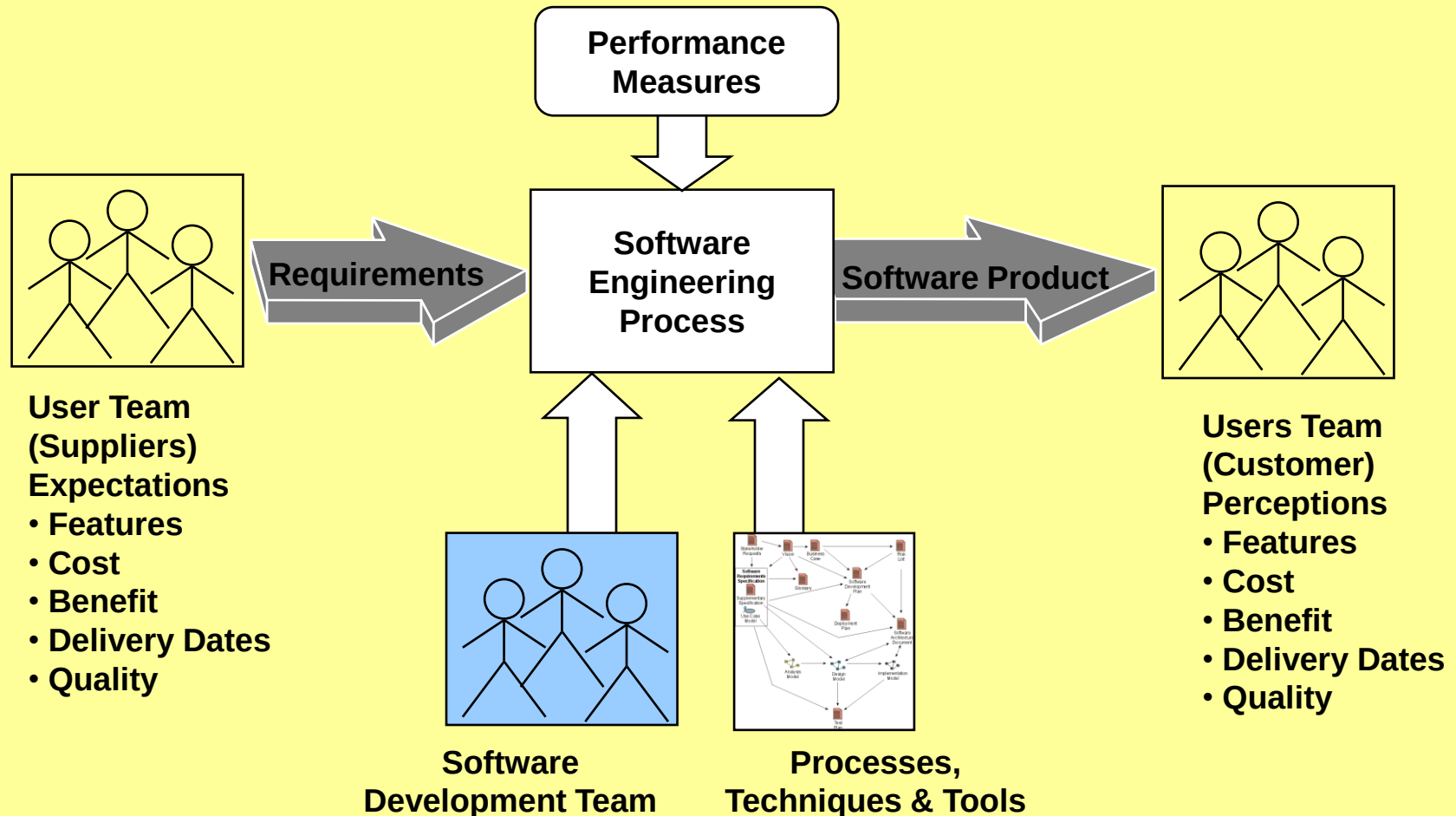
Click on an area of the screen for more information.

RUP *Static Structure: Process Description*

- In classical software engineering, the process primary goal is to identify:
 - who
 - what
 - how
 - when

Rational Unified Process (RUP) – Elements

Software is developed in Teams:



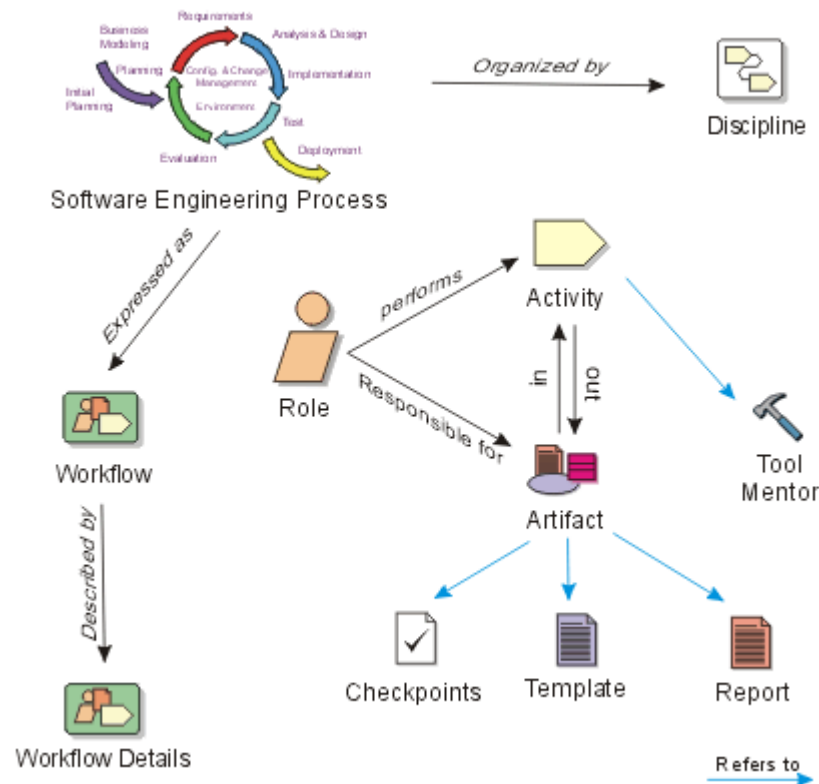
Rational Unified Process (RUP) – Elements

■ In RUP, we have:

- Roles (Worker) : the "who"
- Activities: the "how"
- Artifacts: the "what"
- Workflows: the "when"
- Disciplines: the "Container" for "who, what, how, when"

Rational Unified Process (RUP) -Elements

The RUP Elements : Role, Activities, Artifacts, Workflow & Disciplines



RUP Element - Roles

- The *Role* defines the behavior and responsibilities of the engineer(s).
- Each role is associated with a set of skills required to perform activities of the role
- An engineer may play the role of an architect for two days, a developer next week, and a tester the week after.
- The responsibilities we assign to a worker includes both to perform a certain set of activities as well as being owner of a set of artifacts.



RUP Element - Roles

Examples of Roles :

System Analyst

The engineer who leads and co-ordinates requirement elicitation and use-case modeling by outlining the system's functionality.



RUP Element - Roles

Examples of Roles (Cont...) :

Designer

The engineer who defines the responsibilities, operations, attributes, and relationships of one or more classes and determines how they should be adjusted to the implementation environment.

Test Designer

The engineer who is responsible for generating the test plan and test model, implementing the test procedures, and evaluating test coverage, results and effectiveness.

RUP Element - Roles

Roles - Categories

Roles are organized in five main categories:

- *Analyst Roles*
- *Developer Roles*
- *Tester Roles*
- *Manager Roles*
- *Production and Support Roles*

RUP Element - Roles

The screenshot displays the Eclipse Process Framework Composer (EPF) application. The title bar reads "Eclipse Process Framework Composer - C:\epf-composer\library". The menu bar includes "File", "Edit", "Search", "Configuration", "Window", and "Help". The toolbar contains various icons for file operations and navigation. The left sidebar, titled "Configuration", shows a tree view of the project structure: "OpenUPBasic" > "Role Sets" > "Roles" > "Any Role". The main content area, titled "Content", displays the configuration for the "Any Role".

Role: Any Role

This role is played by any one in a team in order to perform general tasks.
Role Sets: [Roles](#)

[Expand All Sections](#) [Collapse All Sections](#)

Relationships

A diagram shows "Any Role" (represented by a person icon) performing a "Request Change" (represented by a document icon). The relationship is labeled "performs".

Additionally Performs:

- [Assess Results](#)
- [Manage Iteration](#)

Modifies:

- [Work Items List](#)

[Back to top](#)

Main Description

This role performs task that anyone in team can perform, such as:

- accessing artifacts for development and maintenance in the configuration control system
- submitting change requests for the project
- participating in assessments and reviews
- volunteering for doing work in a given iteration

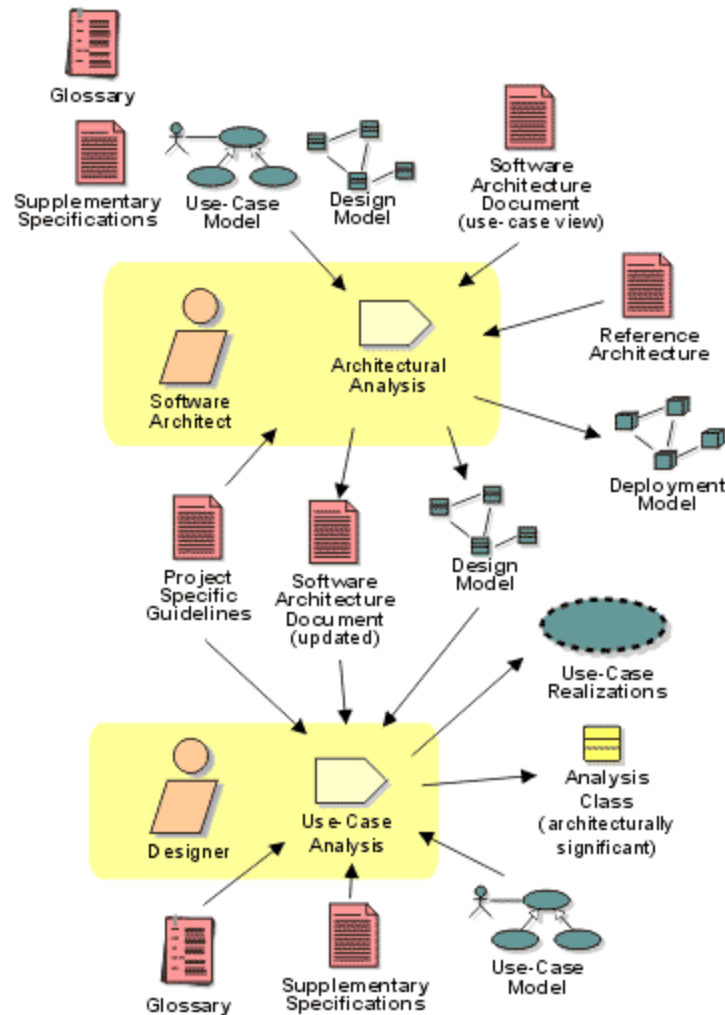
[Back to top](#)

This program and the accompanying materials are made available under the [Eclipse Public License v1.0](#) which accompanies this distribution.

View copyright information here: [OpenUP Copyright](#)

RUP Element - Roles

People and Roles



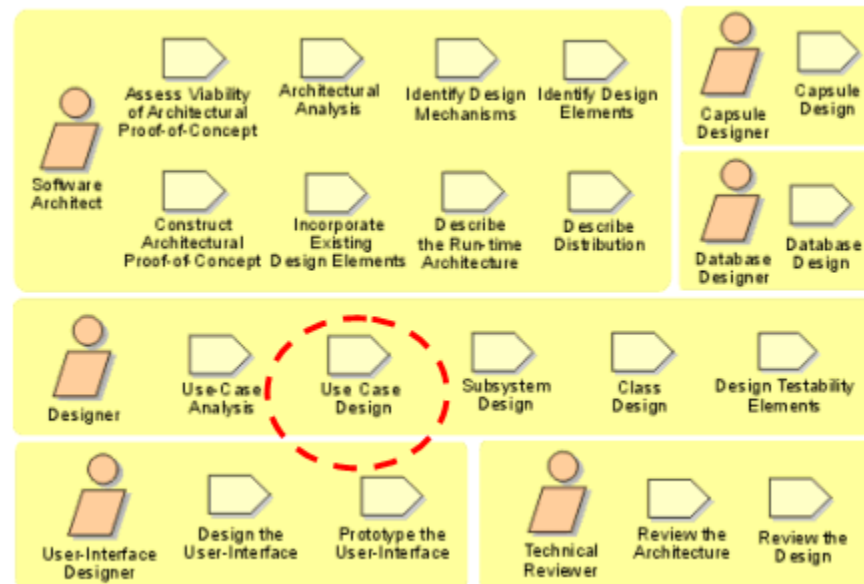
RUP Element - Roles

Discipline	Breadth role	Depth role
Business Modeling	Business Process Analyst Discovers all business use cases.	Business Designer Details a single set of business use cases.
Requirements	Systems Analyst Discovers all requirement use cases.	Requirements Specifier Details a single set of requirement use cases.
Analysis and Design	Software Architect Decides on technologies for the whole solution.	Designer Details the analysis and design for a single set of use cases.
Implementation	Integrator Owns the build plan that shows what classes will integrate with one another.	Implementer Codes a single set of classes or a single set of class operations.
Test	Test Manager Ensures that testing is complete and conducted for the right motivators.	Test Designer Implements automated portions of the test design for the iteration.
	Test Analyst Selects what to test based on the motivators.	Tester Runs a specific test.
	Test Designer Decides what tests should be automated vs. manual and creates automations.	
Deployment	Deployment Manager Oversees deployment for all deployment units.	Tech Writer, Course Developer, Graphic Artist Create detailed materials to ensure a successful deployment.
Project Management	Project Manager Creates the business case and a coarse-grained plan; makes go / no go decisions.	Project Manager Plans, tracks, and manages risk for a single iteration. (<i>Note that this discipline has only one role. Assigning the depth view to a project coordinator can provide relief for overburdened project managers.</i>)
Environment	Process Engineer Owns the process for the project.	Tool Specialist Creates guidelines for using a specific tool.
Configuration and Change Mgt	Configuration Manager Sets up the CM environment, policies, and plan.	Configuration Manager Creates a deployment unit, reports on configuration status, performs audits, and so forth.
	Change Control Manager Establishes a change control process.	Change Control Manager Reviews and manages change requests.
(Again, note that breadth and depth roles are assigned to the same people in this discipline; assistant or associate managers in the depth roles would be helpful.)		

RUP Element - Activities

- An activity of a specific worker is a unit of work that an individual in that role may be asked to perform.
- The activity has a clear purpose, usually expressed in terms of creating or updating some artifacts, such as a model, a class, a plan.
- Every activity is assigned to a specific worker. The granularity of an activity is generally a few hours to a few days, it usually involves one worker, and affects one or only a small number of artifacts.
- An activity should be usable as an element of planning and progress; if it is too small, it will be neglected, and if it is too large, progress would have to be expressed in terms of an activity's parts.

RUP Element - Activities



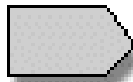
RUP Element - Activities

Examples of Activities:

- Plan an iteration: Project Manager
- Find use cases and actors: System Analyst
- Review the design: Design Reviewer
- Execute performance test: Performance Tester



**Use-Case
Analysis**



**Identify
Design Elements**



**Implement
Design Elements**

Examples of Activities

RUP Element - Activities

An activity has steps and every step fall into one of the following categories:

1. **Thinking Steps:** understand the task, identify the input artifacts, and formulate the outcome.
2. **Performing Steps:** create/update the artifacts
3. **Reviewing Steps:** review/inspect the result based on a reference document.

RUP Element - Activities

Example - Activities broken into steps

Activity: Find Use Cases and Actors has the following steps

1. Find Actors.
 2. Find use cases.
 3. Describe how actors and use cases interact.
 4. Package use cases and actors.
 5. Present the use-case model in use-case Diagrams.
 6. Develop a survey of the use-case model.
 7. Evaluate your results.
- Thinking Steps
- Performance steps
- Reviewing Step

RUP Element - Artifacts

- What is an artifact? An *artifact* is a piece of information that is produced, modified, or used by a process.



- Artifacts are the things the project produces or uses while progressing towards the product final delivery, "drop-off", or "GA: General Availability".
- Artifacts are used as input by workers to perform an activity, and are the result or output of such activities.
- In object-oriented design terms, as activities are operations on an active object (the worker), artifacts are the parameters of these activities.
- An operation/method will have the **input parameters** and **return result (output)**

RUP Element - Artifacts

Artifacts may take various shapes or forms:

- A model, such as the Use-Case Model or the Design Model
- A model element, i.e. an element within a model, such as a class, a use case or a subsystem
- A document, such as Business Case or Software Architecture Document
- Source code
- Executables

RUP Element - Artifacts

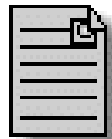
Distinguish between

1. The owner of the artifact
2. The user of the artifact

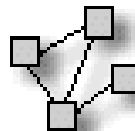
RUP Element - Artifacts

Examples of Artifacts:

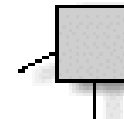
- A design model in UML stored in Rational XDE
- A project plan stored in Microsoft Project
- A defect stored in Rational ClearQuest
- A project requirement database in Rational Requisite Pro



**Software
Architecture
Document**



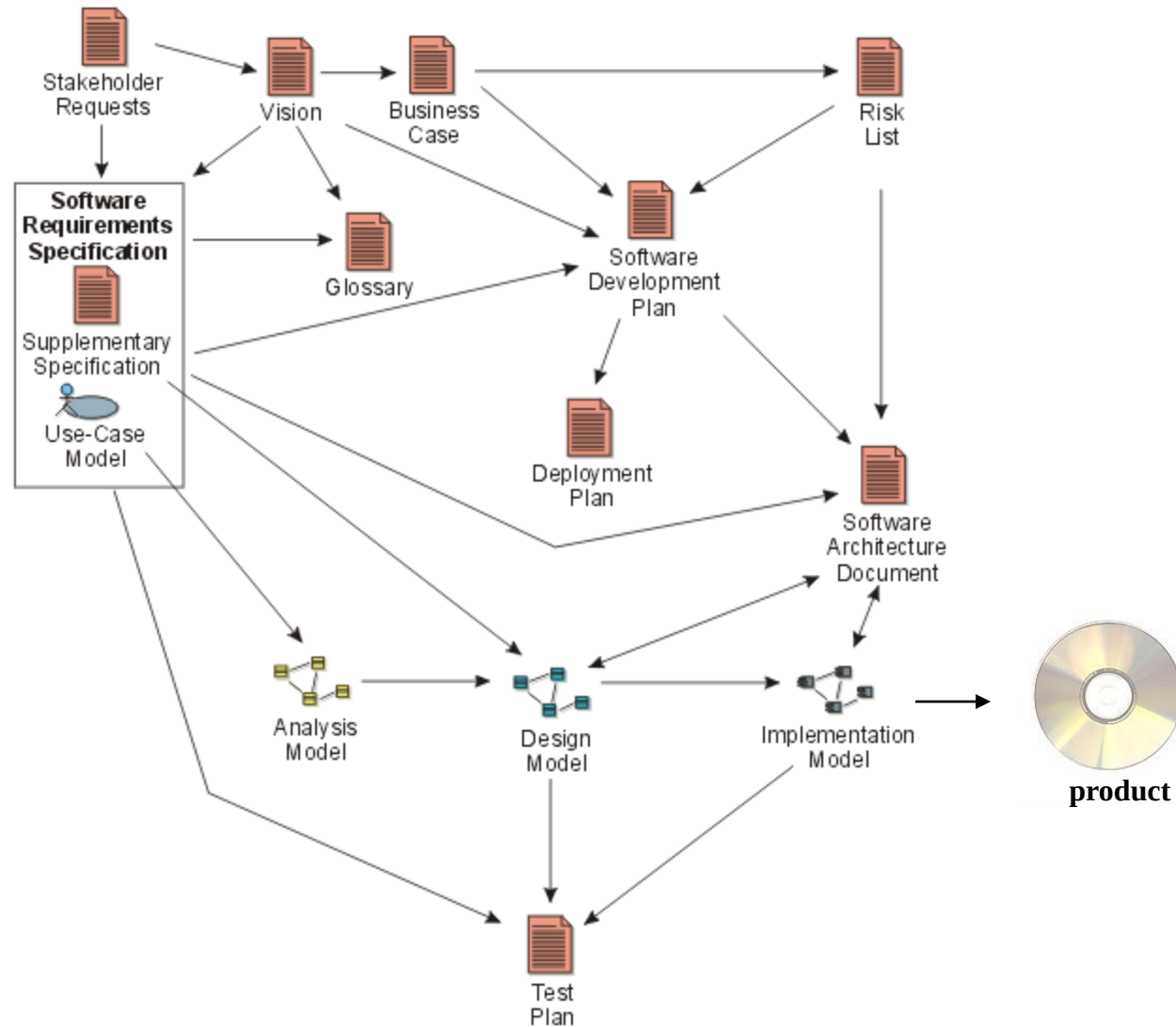
**Implementation
Model**



**Implementation
File**

Examples of RUP Artifacts

RUP Artifacts : Overview



RUP Element - Artifacts

You may associate a **Report** with an **Artifact**

- A Report extracts information about model and model elements from a tool.
- One example for the user of a Report is to present an artifact or a set of artifacts for a review.
- Reports very useful when generating inspection/review packages

RUP Element - Artifacts

RUP's **artifacts** are organized into **information sets** that are aligned with the core **disciplines**.

The Information sets are:

- **Management Set** - artifacts related to software business and to the management of the Project :
 - Planning Artifacts : business case, Software development plan , etc.
 - Operational Artifacts : release notes, deployment documents, defects, etc.
- **Requirements Set** - artifacts related to the definition of software system to be developed
 - The Vision Document
 - Requirements in the form of stakeholders' needs and use case models

RUP Element - Artifacts

Information Sets (Cont...)

- **Design Set :**

- Design Model
- Architecture Description

- **Implementation Set:**

- Source Code and Executables
- data files

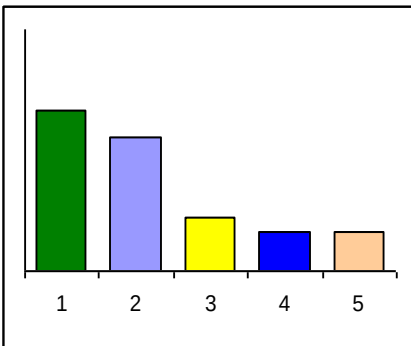
- **Deployment Set:**

- Installation and Training Materials
- User Documentation

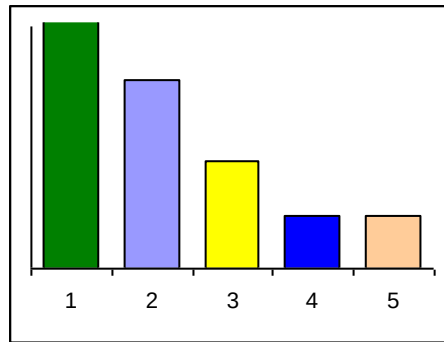
RUP Element – Growing the Information Set

Phases

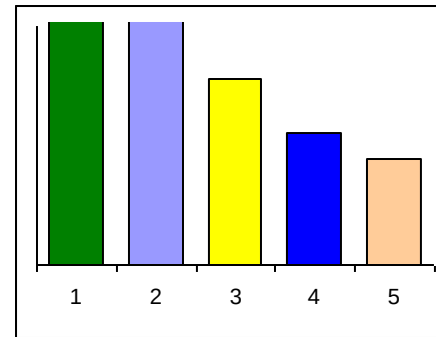
Inception



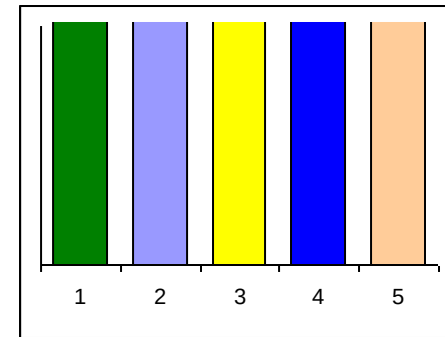
Elaboration



Construction



Implementation



	Management Set
	Requirement Set
	Design Set
	Implementation Set
	Deployment Set

RUP Element - Disciplines

Disciplines as containers for the activities:

RUP has nine core disciplines which are divided into six Technical Disciplines and three supporting Disciplines.

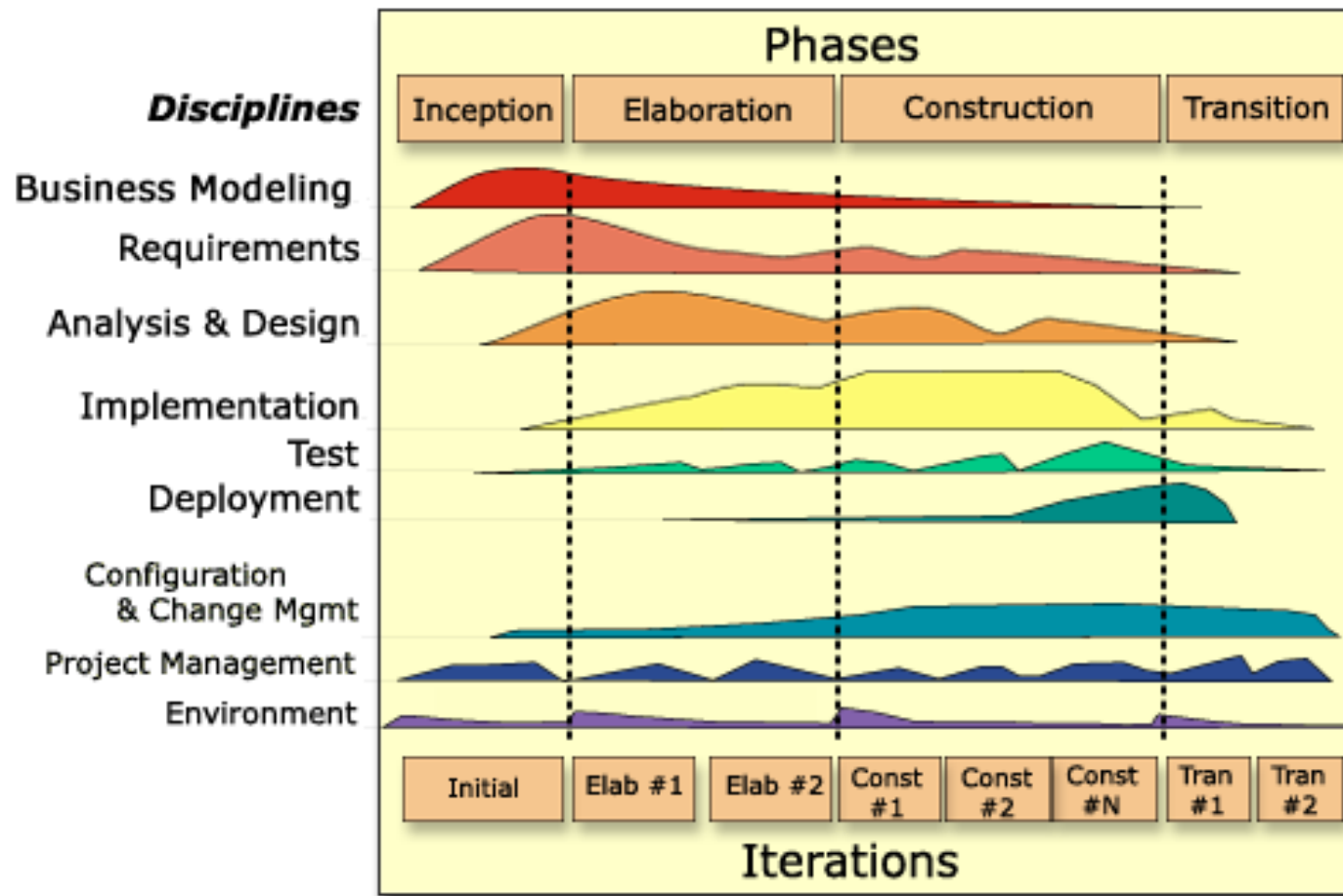
Technical Disciplines :

1. Business modeling discipline
2. Requirement Discipline
3. Analysis and design Discipline
4. Implementation Discipline
5. Test Discipline
6. Deployment Discipline

Supporting Disciplines:

1. Project Management Discipline
2. Environment Discipline
3. Change & Configuration Mgt

RUP Element – Disciplines



RUP Element - Disciplines

RUP – Technical Discipline	Brief Description
Business Modeling	The purpose of the Business Modeling discipline is to: • Understand the structure and the dynamics of the organization in which a system is to be deployed • Understand current problems in the target organization and identify improvement potential • Ensure that customers, end users, and developers have a common understanding of the target organization • Derive the system requirements needed to support the target organization
Requirements	The purpose of the Requirements discipline is to: • Establish and maintain agreement with the customers and other stakeholders on what the system should do • Provide system developers with a better understanding of the system requirements • Define the boundaries of the system • Provide a basis for planning the technical contents of iterations and for estimating the cost and time to develop the system

RUP Element - Disciplines

RUP - Technical Discipline	Brief Description
Analysis and Design	The purpose of the Analysis and Design discipline is to: ▪ Transform the requirements into a design ▪ Evolve a robust architecture for the system ▪ Adapt the design to match the implementation environment
Implementation	The purpose of the Implementation discipline is to: ▪ Define the organization of the implementation ▪ Implement the design elements ▪ Unit test the implementation ▪ Integrate the results produced by individual implementers (or teams), resulting in an executable system

RUP Element - Disciplines

RUP Technical Discipline	Brief Description
Test	The purpose of the Test discipline is to: ▪ Find and document defects in software quality ▪ Provide general advice about perceived software quality ▪ Prove the validity of the assumptions made in design and requirement specifications through concrete demonstration ▪ Validate that the software product functions as designed ▪ Validate that the software product functions as required (that is, the requirements have been implemented appropriately)
Deployment	The purpose of the Deployment discipline is to: ▪ Ensure that the software product is available for its end users

RUP Element - Disciplines

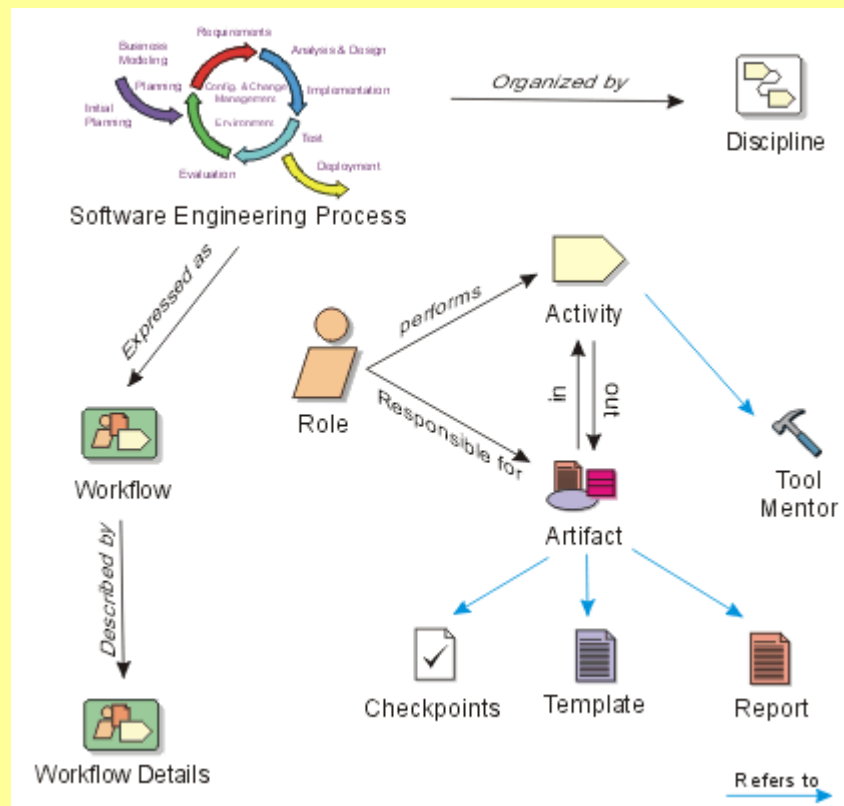
RUP Supporting Discipline	Brief Description
Configuration and Change Management	The purpose of the Configuration and Change Management discipline is to: ▪Identify configuration items ▪Restrict changes to those items ▪Audit changes made to those items ▪Define and manage configurations of those items
Project Management	The purpose of the Project Management discipline is to: ▪Manage a software-intensive project ▪Plan, staff, execute, and monitor a project ▪Manage risk
Environment	The purpose of the Environment discipline is to: Provide the software development organization with the software development environment—both processes and tools—that will support the development team. This includes configuring the process for a particular project, as well as developing guidelines in support of the project.

RUP Element - Workflows

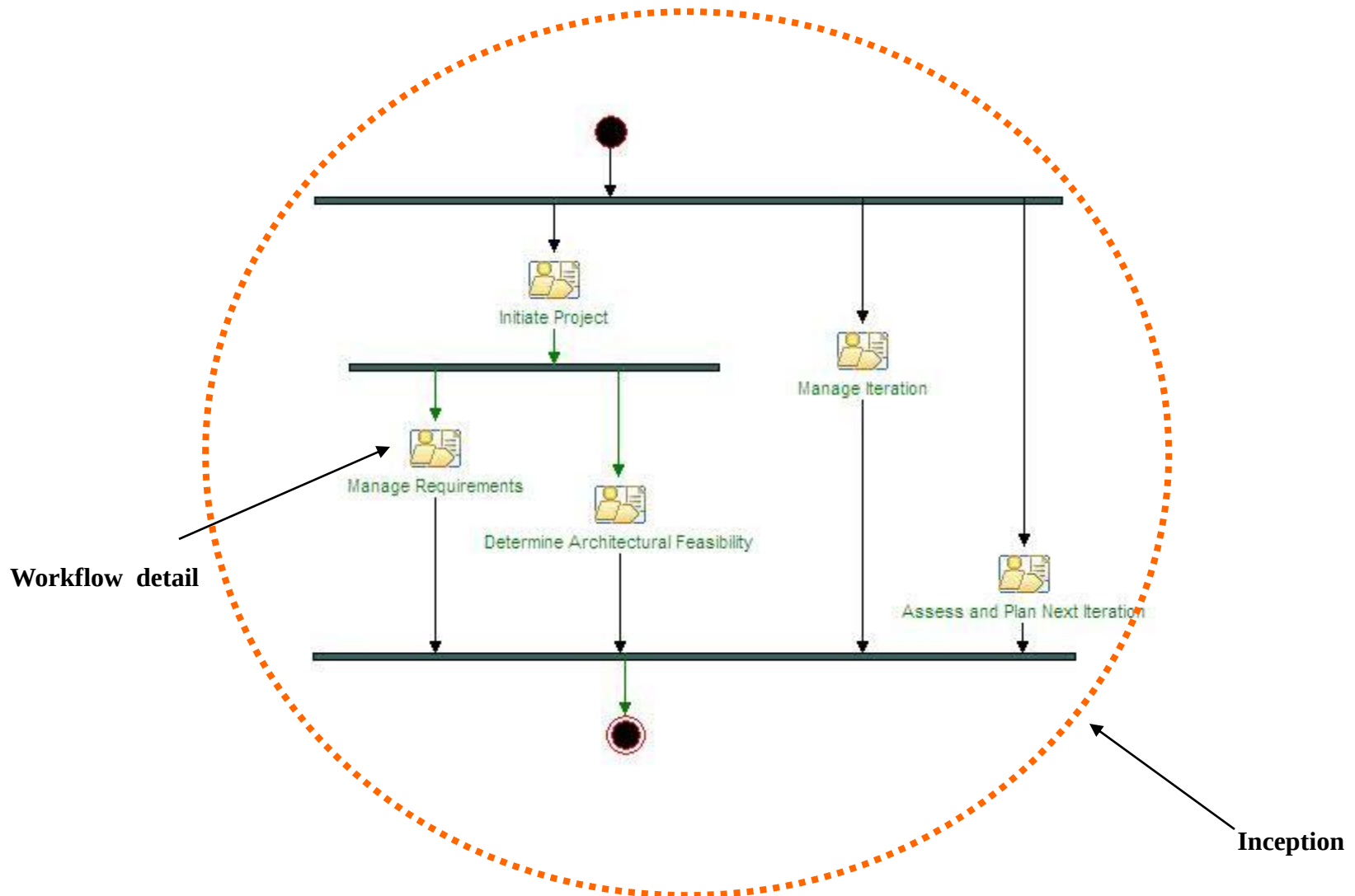
- How to capture the sequences of activities and interactions between the roles?
- A *workflow* is a sequence of activities that produces a result.
- In UML terms, a workflow is expressed as
 - a sequence diagram
 - a collaboration diagram
 - Etc.

RUP Element - Workflows

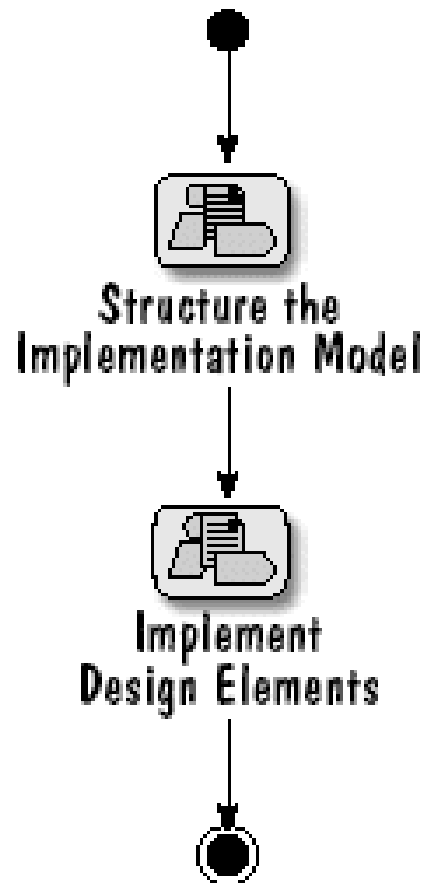
- The Rational Unified Process includes three types of workflow:
 - Core Workflows, associated to each discipline
 - Workflow Details, to refine core workflow
 - Iteration Plans: what happens in typical iteration



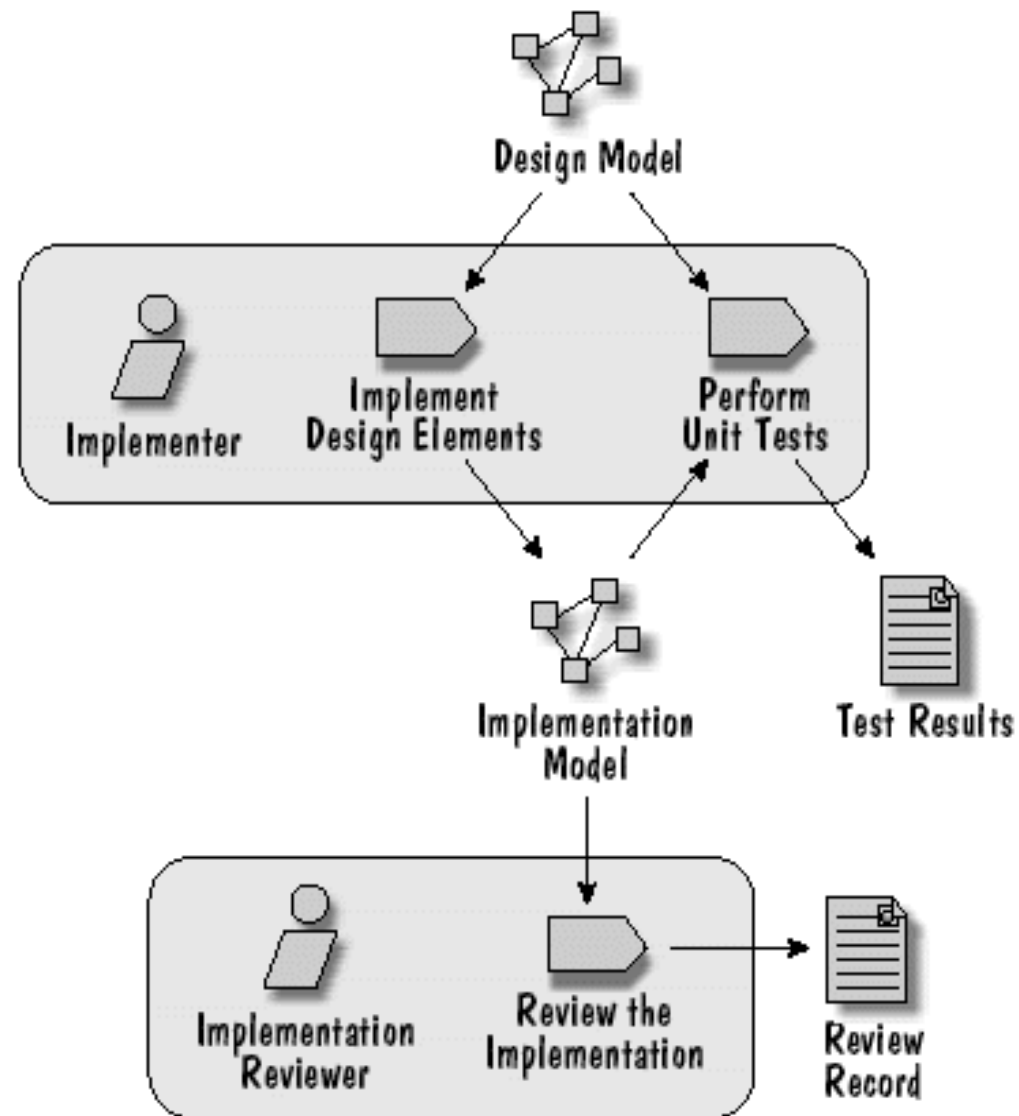
RUP Element - Workflows



RUP Element - Workflows



Implementation Discipline Workflow



RUP – Additional Process Elements

- Why Additional Process Elements?
 - To enhance the primary elements
 - Better Guidance to practitioners

- The Additional Process Elements of RUP are
 - Guidelines
 - Templates
 - Tool Mentors
 - Concepts

RUP – Additional Process Elements

Guidelines

- Guidelines are rules, recommendations , or heuristics that support artifacts, activities and steps.
- Generally, guidelines describe
 - the quality of artifacts
 - techniques to create the artifacts

RUP – Additional Process Elements

Types of Guidelines

- The following are the types of Guidelines:
 - Work Guidelines
 - For example, how to conduct reviews
 - Programming; pair programming
 - Coding Styles; naming convention, etc.
 - Artifact Guidelines
 - Language specific coding-styles
 - Artifact's completion check points
 - Modeling guidelines
 - User-Interface Guidelines

RUP – Additional Process Elements

Templates

Templates

- Think about MS-Word template ...
- Template are models of artifacts that are used to create the artifact
- Usually we end up customizing these templates; adding your name, phone number, company name, etc.

RUP – Additional Process Elements

Tool Mentors

Tool mentors

- Guidelines and activities are general
- Whereas, **Tool Mentors** will show you how to perform a set of process steps using a specific tool
 - Rational Rose
 - ClearCase
 - Etc.

RUP – Additional Process Elements

Concepts

- Iterations and phase are examples of key concepts introduced into the process
- Attached to disciplines

RUP – A Process Framework

- RUP is an open process framework based on an industry standard called SPEM (Software Process Engineering Metamodel)
- RUP is a process framework that can be tailored to specific organization or project needs.

• RUP's elements like roles, activities, artifacts, guidelines, concepts, etc. are the elements that can be added/replaced to adapt the process to fit into the needs of the software development organization.

